

Proyecto de Sistema de Control y Medición de Tráfico

Entrega 1

1st Karol Torres Vides
Carrera de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá D.C., Colombia
torres_kdayan@javeriana.edu.co

2nd Mateo David Guerra Moreno
Carrera de Ingeniería de Sistemas
Pontificia Universidad Javeriana
Bogotá D.C., Colombia
mdavid.guerram@javeriana.edu.co

Resumen—Este documento presenta el diseño y el planteamiento inicial de un sistema distribuido para el control y la medición del tráfico urbano. La solución se organiza en tres nodos principales que cooperan mediante ZeroMQ para adquirir eventos de sensores, distribuirlos, analizarlos, persistirlos y ejecutar acciones de control sobre semáforos. El informe consolida los modelos del sistema, los diagramas de diseño solicitados, la estrategia de tolerancia a fallos ante la caída de la base de datos principal, la configuración inicial de recursos, las reglas de operación, los tipos de consulta de usuario, el protocolo de pruebas y la estrategia para la obtención de métricas de desempeño. Adicionalmente, se resume el estado de implementación alcanzado para la primera entrega y se describe el rol del código fuente actualmente desarrollado.

Index Terms—Sistemas distribuidos, ZeroMQ, tolerancia a fallos, replicación, control de tráfico, semáforos inteligentes.

I. INTRODUCCIÓN

El proyecto propone una plataforma distribuida para la gestión inteligente del tráfico urbano en una ciudad modelada como una cuadrícula de intersecciones. En cada intersección se simulan sensores de tráfico capaces de producir eventos periódicos sobre volumen vehicular, velocidad promedio y condiciones de congestión. Estos eventos son transportados mediante patrones de comunicación síncronos y asíncronos, procesados por un servicio de analítica y utilizados para tomar decisiones sobre la red de semaforización.

La solución se diseñó para cumplir dos objetivos principales. El primero es ofrecer procesamiento distribuido desacoplado, de forma que la generación de datos, el análisis y la persistencia no dependan de una única máquina. El segundo es mantener la operación ante fallos parciales, especialmente cuando el nodo que contiene la base de datos principal deja de responder. Para ello se introduce una réplica en el PC2 y un mecanismo de detección y conmutación que preserve la continuidad del servicio.

II. MODELOS DEL SISTEMA

II-A. Modelo arquitectónico

El sistema se distribuye en tres computadores con responsabilidades diferenciadas:

- **PC1:** aloja los sensores de tráfico y el broker ZeroMQ. Su función es generar eventos y desacoplar la producción de la etapa de análisis.
- **PC2:** contiene el servicio de analítica, el controlador de semáforos, la base de datos réplica y el gestor de resiliencia. Este nodo concentra el procesamiento operacional del sistema.
- **PC3:** alberga la base de datos principal y el servicio de monitoreo y consulta, desde donde el usuario consulta información histórica o puntual y puede emitir instrucciones de prioridad manual.

Desde el punto de vista arquitectónico, la solución sigue una estructura por capas y por responsabilidades. La capa de adquisición está representada por los sensores; la capa de mensajería por el broker ZMQ; la capa de decisión por el servicio de analítica y las reglas; la capa de actuación por el controlador de semáforos; y la capa de persistencia y operación por las bases de datos y el servicio de monitoreo.

II-B. Modelo de interacción

El sistema combina interacciones síncronas y asíncronas para responder a la naturaleza de cada operación.

- **PUB/SUB:** se emplea entre sensores, broker y analítica. Este patrón permite desacoplar productores y consumidores y evita bloquear la producción de eventos cuando el procesamiento toma más tiempo.
- **PUSH/PULL:** se utiliza desde analítica hacia las bases de datos y hacia el control de semáforos. Su objetivo es despachar trabajo de forma asíncrona para mantener la continuidad del flujo.
- **REQ/REP:** se usa en las operaciones de monitoreo y consulta, donde sí existe una solicitud explícita del usuario y una respuesta esperada del sistema.

La interacción está orientada a eventos. Cada medición producida por un sensor se convierte en un mensaje que atraviesa el broker, es interpretado por analítica, se almacena y, si las reglas lo ameritan, produce una acción de control sobre un semáforo. Cuando el usuario realiza una consulta o fuerza

una prioridad, la interacción ocurre en forma de solicitud-respuesta.

II-C. Modelo de fallos

En el sistema se asume la posibilidad de fallos parciales. No todos los componentes caen al mismo tiempo ni de la misma manera, por lo que el diseño debe contemplar detección, aislamiento y recuperación.

II-C1. Clasificación de fallos:

- **Crash faults:** caída completa de un nodo, especialmente del PC3 donde reside la base de datos principal.
- **Fallos de comunicación:** pérdida de conectividad o indisponibilidad temporal de sockets de mensajería.
- **Fallos por latencia:** mensajes entregados con retraso significativo, afectando el tiempo de reacción del sistema.
- **Fallos de consistencia:** divergencia entre la base de datos principal y la réplica si no se controla adecuadamente la replicación.

II-C2. Estrategias de resiliencia: Para mitigar dichos fallos se adoptan los siguientes mecanismos:

- **Replicación activa:** cada evento procesado por analítica se envía tanto a la base de datos principal como a la réplica.
- **Health check:** el gestor de resiliencia verifica periódicamente la disponibilidad del nodo principal.
- **Failover automático:** ante una falla del PC3, las operaciones se redirigen a la réplica en PC2.
- **Reconexión dinámica:** el sistema intenta restablecer el nodo principal cuando se recupera.
- **Idempotencia lógica:** las operaciones críticas de persistencia se diseñan de forma que un reintento no produzca inconsistencias graves.

II-D. Modelo de seguridad

Aunque el proyecto se desarrolla en un entorno académico y con datos simulados, el modelo de seguridad se incorpora desde la fase de diseño. Las principales consideraciones son las siguientes:

- **Confidencialidad:** el acceso al servicio de monitoreo y consulta debe restringirse a usuarios autorizados. En una versión extendida, esto puede representarse mediante autenticación de operadores y validación de comandos de prioridad manual.
- **Integridad:** los mensajes intercambiados entre componentes deben conservar su estructura y contenido. Esto implica validar el formato de los eventos, verificar campos obligatorios y rechazar mensajes incompletos o mal formados.
- **Disponibilidad:** el diseño privilegia la continuidad del servicio mediante replicación, heartbeats y failover. En este proyecto la disponibilidad es el atributo más sensible, pues la utilidad del sistema depende de seguir operando aun ante fallos parciales.
- **Control de acceso:** las acciones manuales, como forzar una prioridad semafórica, deben limitarse a roles de operador y quedar registradas para auditoría.
- **Trazabilidad:** los logs de los servicios constituyen un mecanismo de seguimiento para identificar eventos, decisiones, fallos y acciones de recuperación.

II-E. Modelo de McCumber

El cubo de McCumber permite organizar la seguridad en tres dimensiones: objetivos de seguridad, estados de la información y salvaguardas.

II-E1. Objetivos de seguridad:

- **Confidencialidad:** evitar el acceso no autorizado al módulo de monitoreo y a los comandos de prioridad.
- **Integridad:** preservar la exactitud de los eventos de sensores, las reglas aplicadas y los estados almacenados en las bases de datos.
- **Disponibilidad:** mantener la operación del sistema ante fallos del nodo principal o problemas de comunicación.

II-E2. Estados de la información:

- **En tránsito:** mensajes ZeroMQ entre sensores, broker, analítica, semáforos y bases de datos.
- **En almacenamiento:** eventos históricos, comandos ejecutados y estados de semáforos guardados en la base principal o en la réplica.
- **En procesamiento:** evaluación de reglas, detección de congestión y generación de acciones de control.

II-E3. Salvaguardas:

- **Tecnología:** validación de formatos, manejo de timeouts, replicación y monitoreo de disponibilidad.
- **Políticas y procedimientos:** definición de roles para acciones manuales, protocolo de recuperación ante fallos y control de pruebas.
- **Factores humanos:** uso responsable del servicio de monitoreo, revisión de logs y trazabilidad de acciones especiales.

En el contexto del proyecto, el cubo de McCumber ayuda a justificar por qué la seguridad no se limita a proteger información, sino también a garantizar la continuidad operacional y la correcta interpretación de los eventos de tráfico.

III. DISEÑO DEL SISTEMA

III-A. Diagrama de despliegue

La Figura 1 presenta la distribución física del sistema. Se distinguen los artefactos desplegados en cada nodo, así como los patrones de comunicación entre ellos. La decisión de separar PC1, PC2 y PC3 permite distribuir carga, desacoplar responsabilidades y representar explícitamente el mecanismo de tolerancia a fallos solicitado.

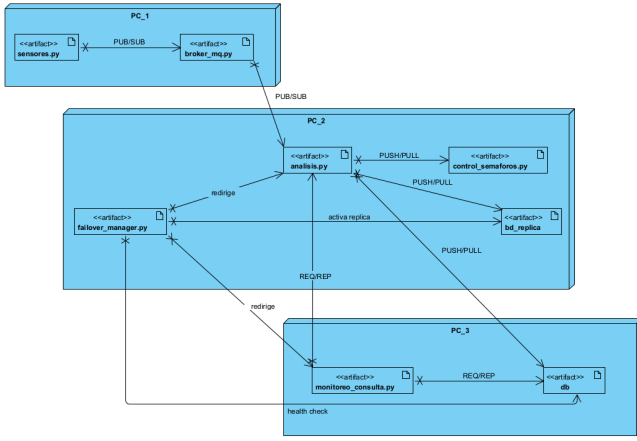


Figura 1: Diagrama de despliegue del sistema distribuido de tráfico.

Descripción del diagrama. En PC1 se generan y distribuyen los eventos mediante `sensores.py` y `broker_mq.py`. En PC2 se realiza el procesamiento operativo con `analisis.py`, el control de semáforos, la base de datos réplica y el componente `failover_manager.py`. En PC3 se localizan la base de datos principal y el módulo de monitoreo y consulta. Las flechas etiquetadas muestran el uso de PUB/SUB, PUSH/PULL y REQ/REP, así como las acciones de redirección y verificación de salud.

III-B. Diagrama de componentes

La Figura 2 muestra la descomposición lógica del sistema en componentes y subsistemas. A diferencia del despliegue, aquí el énfasis está en las responsabilidades funcionales y los contratos de interacción entre bloques.

Descripción del diagrama. El subsistema del PC1 contiene los sensores y el broker. El subsistema del PC2 incorpora el servicio de analítica, el controlador de semáforos, la base de datos réplica y el gestor de resiliencia. El subsistema del PC3 integra la base de datos principal y el servicio de monitoreo y consulta. Las interfaces de publicación, distribución, comandos, persistencia y consultas reflejan los intercambios principales del sistema. El gestor de resiliencia supervisa la base de datos principal y redirige operación y consultas hacia la réplica cuando es necesario.

III-C. Diagrama de clases

La Figura ?? detalla la estructura interna del software. Se incluyen las clases del dominio, de control, de persistencia y de configuración. Este diagrama resulta útil para justificar la organización del código fuente y la separación de responsabilidades.

Descripción del diagrama. La jerarquía de sensores se organiza a partir de una clase base `Sensor`, especializada en `CamaraSensor`, `EspiraSensor` y `GPSSensor`. La clase `EventoTráfico` representa la unidad de información intercambiada entre componentes. `BrokerMQ` recibe y redistribuye eventos, mientras que `ServicioAnalitica`

procesa dichos eventos, evalúa reglas y genera acciones hacia `ControlSemáforos`. La persistencia se abstrae mediante las clases `BD` y `BD_rep`. `ServicioMonitoreo` gestiona consultas y `GestorResiliencia` supervisa la disponibilidad del servicio principal.

III-D. Diagramas de secuencia

Para evidenciar el comportamiento dinámico del sistema se presentan dos secuencias: una de operación normal y otra de operación ante falla del nodo principal.

III-D1. Secuencia de procesamiento normal: Descripción del diagrama. Un sensor publica un evento al broker, que lo reenvía a analítica. Luego, analítica evalúa reglas, persiste el evento en la base de datos principal y replica el mismo evento en la base de datos secundaria. Si la condición evaluada corresponde a congestión, el sistema emite un comando al controlador de semáforos para extender la fase verde. En caso contrario, el flujo termina sin acción adicional.

III-D2. Secuencia de tratamiento de falla: Descripción del diagrama. Cuando `ServicioAnalitica` intenta almacenar un evento en la base principal y no obtiene respuesta, notifica el fallo al `GestorResiliencia`. Este verifica la salud del servicio, confirma el timeout y ordena conmutar hacia la base de datos réplica. A partir de ese momento, tanto la persistencia de nuevos eventos como las consultas del servicio de monitoreo se redirigen a la réplica, manteniendo la continuidad de la operación.

IV. INICIALIZACIÓN DEL SISTEMA

La configuración inicial se centraliza en un archivo de parámetros que define direcciones IP, puertos, intersecciones y frecuencia de generación de eventos. Esta decisión facilita la parametrización del sistema para correr en una sola máquina, en máquinas virtuales o en tres equipos físicos.

IV-A. Parámetros base

- Direcciones de red de cada computador: `PC1_IP`, `PC2_IP` y `PC3_IP`.
- Direcciones de sockets ZMQ para cada tipo de comunicación.
- Lista de intersecciones simuladas, por ejemplo `INT_A1`, `INT_A2`, `INT_B1`, `INT_B2`.
- Intervalo entre mediciones de sensores.
- Tiempo base de fase verde para los semáforos.

IV-B. Recursos iniciales del sistema

Para la primera entrega se asume una ciudad mínima con una cuadrícula reducida, suficiente para validar el flujo completo del sistema. La definición inicial de recursos incluye:

- número y tipo de sensores por intersección;
- número de intersecciones modeladas en la cuadrícula;
- número de semáforos asociados a cada intersección;
- puertos y direcciones para los servicios distribuidos;
- tiempos de muestreo y tiempos base de cambio de fase.

Esta parametrización permite ampliar progresivamente el sistema para las pruebas de desempeño sin modificar la lógica principal del código.

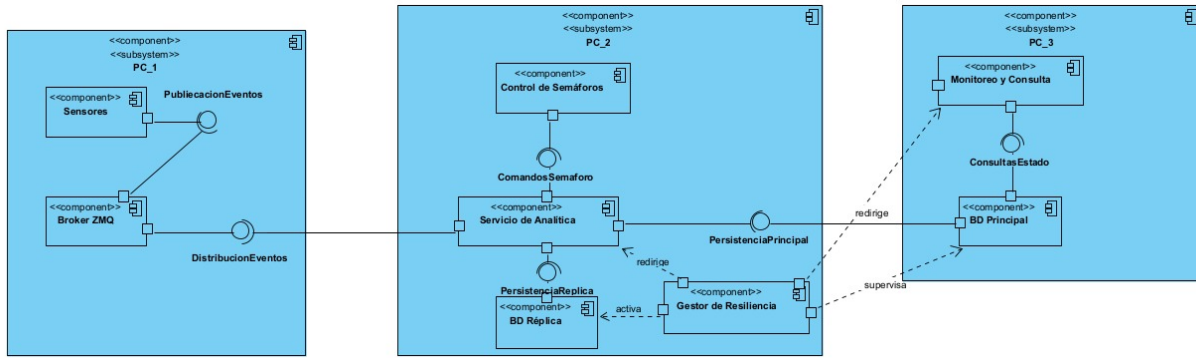


Figura 2: Diagrama de componentes del sistema.

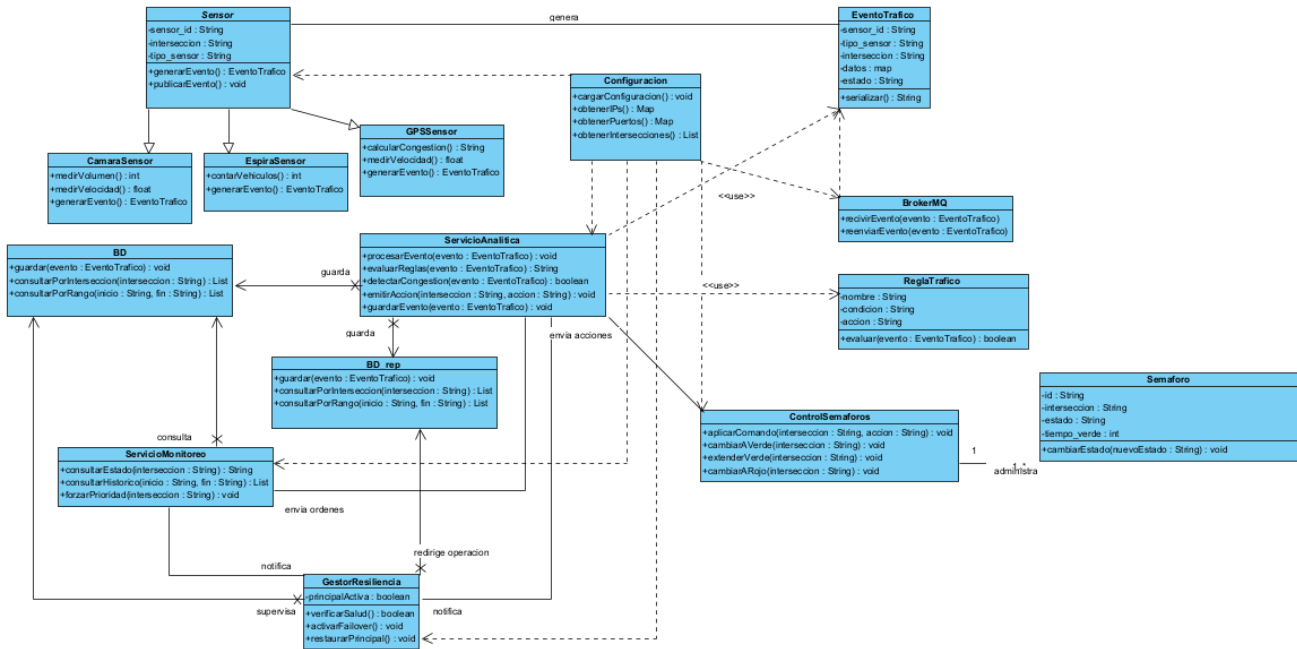


Figura 3: Diagrama de clases del sistema.

V. REGLAS DEL SISTEMA

Las reglas del sistema traducen mediciones de sensores en estados de operación de la red semafórica. En esta primera versión se utilizan reglas simples, suficientes para validar la lógica distribuida.

V-A. Tráfico normal

Se considera tráfico normal cuando la cola es baja, la velocidad promedio es alta y la densidad es moderada. Una formulación de referencia es:

$$Q < 5 \quad \wedge \quad V_p > 35 \quad \wedge \quad D < 20$$

V-B. Congestión

Se considera congestión cuando la longitud de cola aumenta y la velocidad promedio disminuye. Una aproximación de

referencia es:

$$Q > 5 \quad \wedge \quad V_p < 35$$

En este estado, el servicio de analítica puede emitir acciones como extensión de fase verde para aliviar temporalmente la saturación de una vía.

V-C. Priorización

La priorización corresponde a una intervención manual o a una condición especial previamente definida. En la primera entrega se modela principalmente como una orden manual desde el servicio de monitoreo para habilitar el paso preferente, por ejemplo ante la circulación de una ambulancia.

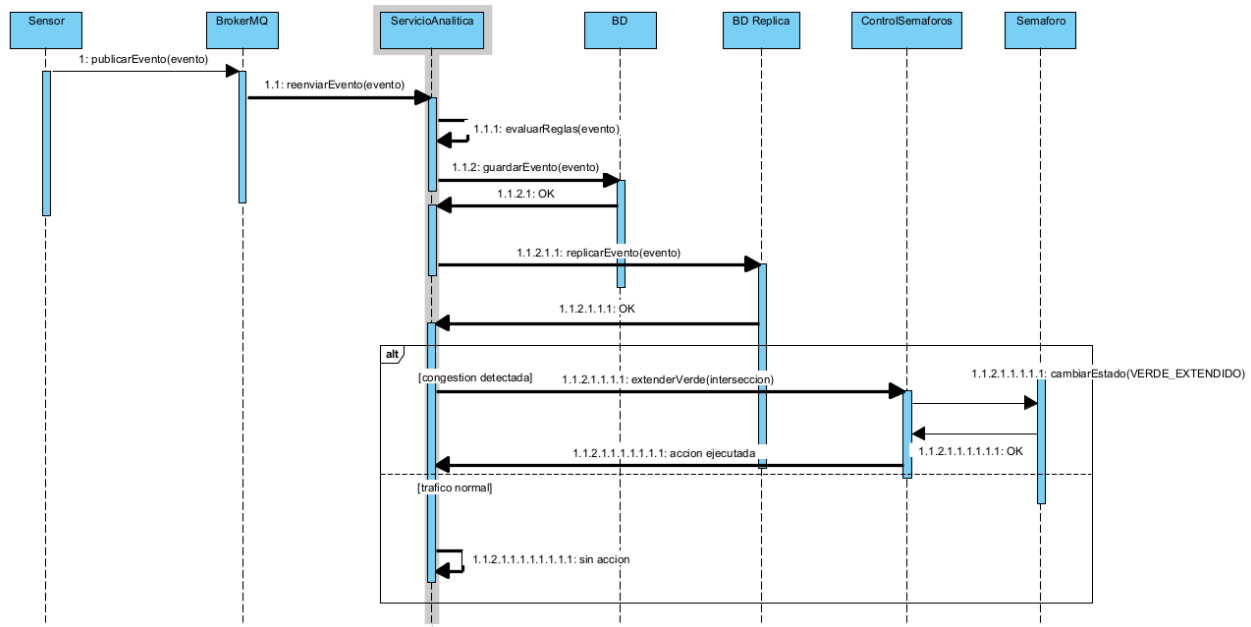


Figura 4: Diagrama de secuencia para el procesamiento normal de un evento.

VI. CONSULTAS DEL USUARIO E INDICACIONES DIRECTAS

El servicio de monitoreo y consulta debe soportar dos categorías de interacción: consulta de información y emisión de órdenes especiales.

VI-A. Consultas

- **Consulta puntual por intersección:** permite conocer el estado reciente de una intersección específica.
- **Consulta histórica por intervalo:** permite revisar eventos y estados en una franja de tiempo, por ejemplo horas pico.
- **Consulta de acciones ejecutadas:** permite verificar cambios de fase y eventos de priorización registrados.

VI-B. Indicaciones directas

- forzar fase verde en una intersección específica;
- habilitar prioridad temporal para una vía;
- restablecer el funcionamiento normal después de una prioridad;
- solicitar el estado de disponibilidad de la base de datos principal y la réplica.

VII. DESCRIPCIÓN DEL CÓDIGO FUENTE IMPLEMENTADO

La primera entrega exige que ya existan servicios de PC1 y PC2, además de la actualización de la base de datos principal en PC3. En el estado actual del proyecto, los archivos principales cumplen las siguientes responsabilidades:

VII-A. *config.py*

Centraliza direcciones IP, puertos, nombres de intersecciones y parámetros globales de simulación. Su objetivo es desacoplar la lógica de los servicios de la configuración de red y del tamaño del escenario.

VII-B. *sensores.py*

Simula la generación de eventos de diferentes tipos de sensor. Construye eventos con marca de tiempo y los publica hacia el broker mediante PUB/SUB. Este archivo representa la capa de adquisición del sistema.

VII-C. *broker_mq.py*

Funciona como intermediario de mensajería. Se suscribe a los tópicos de sensores y redistribuye los eventos al servicio de analítica. Su función principal es desacoplar productores y consumidores.

VII-D. *analisis.py*

Es el núcleo del sistema. Recibe eventos, aplica reglas, determina el estado de tráfico, persiste la información y genera comandos de control para los semáforos. También participa en la gestión de failover al cambiar el destino de persistencia cuando la base principal deja de responder.

VII-E. *control_semaforos.py*

Ejecuta las órdenes emitidas por analítica. Cambia estados de semáforos e imprime las acciones realizadas, lo que facilita la observación del sistema durante las pruebas y la sustentación.

VII-F. *db.py*

Representa la base de datos principal ubicada en el PC3. Recibe eventos desde analítica y los almacena como persistencia principal del sistema.

VII-G. *db_replica.py*

Almacena una copia redundante de los eventos en el PC2. Su presencia permite mantener continuidad operativa y soportar el mecanismo de failover.

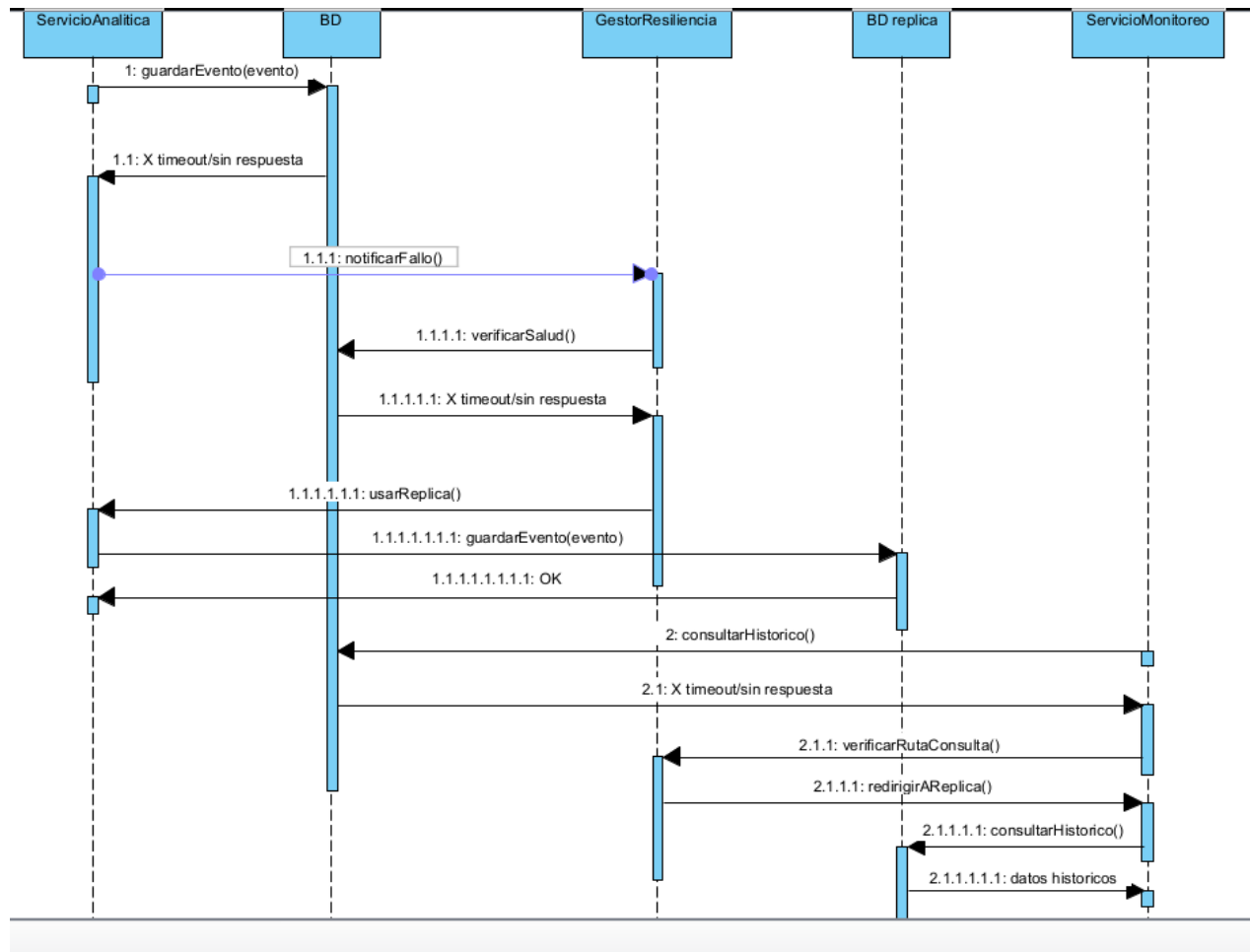


Figura 5: Diagrama de secuencia ante la caída de la base de datos principal.

VII-H. *failover_manager.py*

Monitorea la disponibilidad del nodo principal, detecta fallos por timeout y activa la conmutación hacia la réplica. Este archivo concentra la lógica de health check y redirección.

VII-I. *monitoreo_consulta.py*

Ofrece el punto de contacto con el usuario. Permite consultar el estado del sistema, revisar información histórica y solicitar prioridades manuales. En el diseño completo también debe participar en la redirección de consultas cuando el PC3 no está disponible.

VIII. PROTOCOLO DE PRUEBAS

El protocolo de pruebas se plantea de forma incremental, iniciando con validaciones funcionales, continuando con integración y tolerancia a fallos, y terminando con pruebas de desempeño. Esta estrategia permite aislar problemas tempranamente y construir evidencia de funcionamiento para la sustentación.

VIII-A. *Pruebas funcionales*

- **Caso 1. Generación de eventos:** verificar que los sensores produzcan mensajes correctos en formato y frecuencia.

- **Caso 2. Reenvío por broker:** verificar que los eventos publicados sean entregados a analítica sin pérdidas evidentes.
- **Caso 3. Evaluación de reglas:** validar la clasificación de estados normales y de congestión con valores controlados.
- **Caso 4. Control de semáforos:** verificar la recepción y ejecución de comandos de cambio o extensión de fase.

VIII-B. *Pruebas de integración*

- **Caso 5. Flujo extremo a extremo:** ejecutar los servicios de PC1, PC2 y PC3 simultáneamente y verificar que un evento recorra adquisición, broker, analítica, persistencia y acción.

VIII-C. *Pruebas de tolerancia a fallos*

- **Caso 6. Caída de PC3:** detener la base de datos principal durante la ejecución normal y verificar que el sistema continúe operando con la réplica.
- **Caso 7. Recuperación del nodo principal:** reactivar el servicio principal y comprobar la reconexión y el retorno gradual al estado nominal.

VIII-D. *Pruebas de desempeño*

Las pruebas de desempeño se alinean con el enunciado y se concentran en dos variables dependientes:

1. cantidad de solicitudes almacenadas en la base de datos en un intervalo de 2 minutos;
2. tiempo desde que el usuario solicita una acción hasta que el semáforo ejecuta el cambio correspondiente.

Se evaluarán, como mínimo, los siguientes escenarios:

- **Escenario 1 (carga baja):** un sensor de cada tipo generando datos cada 10 segundos.
- **Escenario 2 (carga media):** dos sensores de cada tipo generando datos cada 5 segundos.

Como extensión para la entrega final, se comparará el diseño base con una variante de broker multihilo para observar el impacto sobre rendimiento y escalabilidad.

IX. OBTENCIÓN DE MÉTRICAS DE DESEMPEÑO

La estrategia de medición se basa en timestamps y conteos observables en logs y bases de datos.

IX-A. Métrica 1: solicitudes almacenadas en 2 minutos

Se contará el número de eventos persistidos en la base de datos durante una ventana fija de dos minutos. Este valor se puede obtener revisando el número de registros escritos o contabilizando líneas válidas en los archivos de almacenamiento.

IX-B. Métrica 2: tiempo de respuesta usuario-semáforo

Se registrará un timestamp cuando el usuario emita una orden desde el servicio de monitoreo y otro cuando el controlador de semáforos confirme la acción ejecutada. La diferencia entre ambos tiempos corresponde a la latencia de respuesta observable.

IX-C. Herramientas de medición

- logs de los servicios distribuidos;
- registros de la base de datos principal y de la réplica;
- scripts de análisis en Python para calcular promedios, máximos y mínimos;
- conteos por escenario de carga para comparar capacidad de procesamiento.

IX-D. Criterios de evaluación

Se considerará que el sistema presenta un comportamiento aceptable si:

- mantiene tiempos de respuesta consistentes al aumentar la carga;
- persiste la mayoría de eventos generados sin pérdidas significativas;
- conserva continuidad de operación cuando falla el nodo principal;
- muestra una mejora o al menos estabilidad al comparar el diseño base con la variante multihilo del broker.